A Proposed Cover-Tree Tableau Calculus for $\mathcal{ALCI}_{RCC5}$ with Eventuality-Aware Signature Blocking

A proof-style note

April 2026

Abstract

This note packages the split-forest / sibling-interface approach into a concrete tableau proposal. The underlying idea is to reason on a *cover tree*: downward edges are immediate PPI-steps, upward edges are immediate PP-steps, while all indirect PP/PPI facts are interpreted through the ancestor relation. Because the ambient root of the cover tree need not coincide with the point at which the input concept is evaluated, the calculus works with a distinguished evaluation node and permits *top extension* when upward eventualities demand new ancestors.

The local part of the calculus handles booleans and the transitive modalities ∃PP, ∃PPI, ∀PP, ∀PPI through ancestor/descendant requests and obligations. The non-tree modalities DR and PO are not materialized by local witness creation; instead, they are discharged by a finite global side-checker built from the sibling-interface descriptors of the split-forest certificate. Blocking is equality of finite rank-d signatures, where $d = \mathrm{md}(C_0)$ and the signature records local labels, eventualities, parent summaries, child multiplicities, sibling-interface summaries, and witness menus.

The note states a concrete calculus, an eventuality-aware blocking condition, and proof-style arguments for termination, soundness, and completeness. The arguments are explicit about one condensed point: the extraction of support-closed witness/interface menus from a model is finite but not written out at maximal low-level detail. Relative to the companion split-forest theorem, the present calculus should be read as a proposed operationalization rather than as an independently verified new theorem.

1 Context and strategy

Wessel's original report already isolates the source of difficulty in $\mathcal{ALCI}_{RCC5}$: models are complete RCC5-labelled graphs rather than ordinary modal trees, and infinitary behaviour is driven by proper-part chains rather than by plain modal branching [1]. The sibling-interface program attacks this by replacing the full complete graph by a weak-EQ split-forest certificate whose open cross-edges are reduced to finite DR/PO choices. The companion note on split-forest certificates proposes a path-consistent disjunctive reconstruction of those open edges by finite-prefix patchwork arguments.

The purpose of the present note is to turn that semantic picture into a tableau-style decision procedure. The guiding principles are the following.

- The tableau keeps only the *cover tree*. It does not materialize indirect PP/PPI edges.
- Formulas ∃PPI.C and ∃PP.C are treated as *eventualities* over the descendant / ancestor order of the cover tree, not as immediate child/parent witnesses.

- Formulas $\exists\mathsf{DR}.C$ and $\exists\mathsf{PO}.C$ are discharged globally by a finite sibling-interface solver, not by local child creation.
- Blocking is not plain type equality. It is equality of finite rank-d signatures containing local labels, eventualities, parent summaries, child multiplicities, sibling-interface summaries, and witness menus.

This makes the calculus hybrid: local tableau saturation plus a finite global side-condition. The side-condition is exactly what the split-forest proof needs anyway; the point of the calculus is to expose it operationally.

2 Preliminaries

Let C_0 be the input concept and let $d := \mathrm{md}(C_0)$ be its modal depth. Let $\mathrm{cl}(C_0)$ be its Fischer–Ladner closure and $\mathrm{Tp}(C_0)$ its finite set of Hintikka types. We write

$$\mathsf{RCC5} := \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}.$$

The nontrivial singleton composition laws used repeatedly are

$$\mathrm{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}, \qquad \mathrm{comp}(\mathsf{DR}, \mathsf{PPI}) = \{\mathsf{DR}\}.$$

We also use the full tractability of disjunctive RCC5 networks: every path-consistent finite disjunctive RCC5 network has an atomic refinement [1, 2].

Remark 2.1. The cover tree is oriented downward by immediate PPI steps. Hence semantic PPI means "some strict descendant" and semantic PP means "some strict ancestor". This is why the input concept is evaluated at a distinguished node and not necessarily at the ambient root of the current tree. The formula $\exists\mathsf{PP}.\top$ is perfectly legitimate: it simply means that the evaluation node must have some strict ancestor above it.

3 Tableau objects

Definition 3.1 (Cover-tree tableau)**.** *A* cover-tree tableau *for C_0 is a tuple*

$$\mathcal{T} = (V, \preceq, \mathsf{par}, \lhd, r_*, \varrho, L, \mathsf{All}^{\uparrow}, \mathsf{All}^{\downarrow}, \mathsf{Req}^{\uparrow}, \mathsf{Req}^{\downarrow}, \mathsf{Need}_{\mathsf{DR}}, \mathsf{Need}_{\mathsf{PO}}, \mathsf{XReq}_{\mathsf{DR}}, \mathsf{XReq}_{\mathsf{PO}})$$

where:

(i) (V, par) *is a finite rooted tree with ambient root ϱ;*

(ii) $r_* \in V$ *is the distinguished* evaluation node*;*

(iii) $\lhd_u$ *is an arbitrary but fixed local total order on the children of each node u;*

(iv) $L(x) \subseteq \mathrm{cl}(C_0)$ *is the local label of x;*

(v) $\mathsf{All}^{\uparrow}(x), \mathsf{All}^{\downarrow}(x), \mathsf{Req}^{\uparrow}(x), \mathsf{Req}^{\downarrow}(x), \mathsf{Need}_{\mathsf{DR}}(x), \mathsf{Need}_{\mathsf{PO}}(x), \mathsf{XReq}_{\mathsf{DR}}(x), \mathsf{XReq}_{\mathsf{PO}}(x) \subseteq \mathrm{cl}(C_0)$ *are finite sets of obligations and requests at x.*

The intended meaning is:

- $C \in \mathsf{All}^{\uparrow}(x)$*: every strict ancestor of* x *must satisfy* C*;*
- $C \in \mathsf{All}^{\downarrow}(x)$*: every strict descendant of* x *must satisfy* C*;*
- $C \in \mathsf{Req}^{\uparrow}(x)$*: some strict ancestor of* x *must satisfy* C*;*
- $C \in \mathsf{Req}^{\downarrow}(x)$*: some strict descendant of* x *must satisfy* C*;*
- $C \in \mathsf{Need}_{\mathsf{DR}}(x)$*: every future* DR*-related witness of* x *must satisfy* C*;*
- $C \in \mathsf{Need}_{\mathsf{PO}}(x)$*: every future* PO*-related witness of* x *must satisfy* C*;*
- $C \in \mathsf{XReq}_{\mathsf{DR}}(x)$*:* x *requires some* DR*-related witness satisfying* C*;*
- $C \in \mathsf{XReq}_{\mathsf{PO}}(x)$*:* x *requires some* PO*-related witness satisfying* C*.*

Remark 3.2. The local child order has no semantic meaning. It is only used to orient sibling-interface descriptors as ordered left/right pairs, exactly as in the split-forest note. Any other fixed local order yields an isomorphic descriptor calculus.

Definition 3.3 (Local clashes)**.** *A node* x *has a* local clash *if one of the following holds:*

(a) $A, \neg A \in L(x)$ *for some concept name* A*;*

(b) $C \in L(x) \cap \mathsf{All}^{\uparrow}(x)$ *and* $\neg C \in L(x)$*, or symmetrically with* $\mathsf{All}^{\downarrow}(x)$*;*

(c) $\mathsf{par}(x) = y$ *and some* $C \in \mathsf{All}^{\uparrow}(x)$ *is incompatible with* $L(y)$ *after boolean saturation;*

(d) some eventuality/request is explicitly contradicted by an already fixed local fact.

A tableau branch is locally clash-free *if no node has a local clash.*

4 Operational rules

The calculus works modulo boolean saturation. Whenever a rule adds a formula to a local label, the standard propositional closure rules are applied immediately.

4.1 Boolean rules

- ($\sqcap$) If $D_1 \sqcap D_2 \in L(x)$ and one of D_1, D_2 is missing, add both to $L(x)$.
- ($\sqcup$) If $D_1 \sqcup D_2 \in L(x)$ and neither disjunct is present, branch and add one chosen disjunct.

4.2 Universal initialization

- ($\forall$PPI-init) If $\forall\mathsf{PPI}.C \in L(x)$ and $C \notin \mathsf{All}^{\downarrow}(x)$, add C to $\mathsf{All}^{\downarrow}(x)$.
- ($\forall$PP-init) If $\forall\mathsf{PP}.C \in L(x)$ and $C \notin \mathsf{All}^{\uparrow}(x)$, add C to $\mathsf{All}^{\uparrow}(x)$.
- ($\forall$DR-init) If $\forall\mathsf{DR}.C \in L(x)$ and $C \notin \mathsf{Need}_{\mathsf{DR}}(x)$, add C to $\mathsf{Need}_{\mathsf{DR}}(x)$.
- ($\forall$PO-init) If $\forall\mathsf{PO}.C \in L(x)$ and $C \notin \mathsf{Need}_{\mathsf{PO}}(x)$, add C to $\mathsf{Need}_{\mathsf{PO}}(x)$.

Definition 4.1 (Local DR/PO type-safety slots)**.** *For every tableau node x, the sets $\mathsf{Need}_{\mathsf{DR}}(x)$ and $\mathsf{Need}_{\mathsf{PO}}(x)$ collect the universal requirements that must be met by future open cross-witnesses of x:*

$$\mathsf{Need}_{\mathsf{DR}}(x) = \{C \in \mathrm{cl}(C_0) : \forall \mathsf{DR}.C \in L(x)\}, \qquad \mathsf{Need}_{\mathsf{PO}}(x) = \{C \in \mathrm{cl}(C_0) : \forall \mathsf{PO}.C \in L(x)\}.$$

Because both DR *and* PO *are symmetric in RCC5, a candidate* DR*-witness edge between tableau points x, y is locally type-safe iff*

$$\mathsf{Need}_{\mathsf{DR}}(x) \subseteq L(y) \qquad \textit{and} \qquad \mathsf{Need}_{\mathsf{DR}}(y) \subseteq L(x),$$

and a candidate PO*-witness edge between x, y is locally type-safe iff*

$$\mathsf{Need}_{\mathsf{PO}}(x) \subseteq L(y) \qquad \textit{and} \qquad \mathsf{Need}_{\mathsf{PO}}(y) \subseteq L(x).$$

These constraints are explicit signature components; they are separate from the existential request slots $\mathsf{XReq}_{\mathsf{DR}}$ and $\mathsf{XReq}_{\mathsf{PO}}$.

4.3 Universal propagation along the cover tree

- ($\forall$PPI-prop) If $C \in \mathsf{All}^{\downarrow}(x)$ and y is a child of x, then add C to $L(y)$ and to $\mathsf{All}^{\downarrow}(y)$.
- ($\forall$PP-prop) If $C \in \mathsf{All}^{\uparrow}(x)$ and x has parent p, then add C to $L(p)$ and to $\mathsf{All}^{\uparrow}(p)$.
- ($\forall$PP-top) If $C \in \mathsf{All}^{\uparrow}(\varrho)$ and later the ambient root is extended upward by a fresh parent p, then C must immediately be inserted into $L(p)$ and into $\mathsf{All}^{\uparrow}(p)$.

4.4 Existential initialization

- ($\exists$PPI-init) If $\exists\mathsf{PPI}.C \in L(x)$ and $C \notin \mathsf{Req}^{\downarrow}(x)$, add C to $\mathsf{Req}^{\downarrow}(x)$.
- ($\exists$PP-init) If $\exists\mathsf{PP}.C \in L(x)$ and $C \notin \mathsf{Req}^{\uparrow}(x)$, add C to $\mathsf{Req}^{\uparrow}(x)$.
- ($\exists$DR-init) If $\exists\mathsf{DR}.C \in L(x)$ and $C \notin \mathsf{XReq}_{\mathsf{DR}}(x)$, add C to $\mathsf{XReq}_{\mathsf{DR}}(x)$.
- ($\exists$PO-init) If $\exists\mathsf{PO}.C \in L(x)$ and $C \notin \mathsf{XReq}_{\mathsf{PO}}(x)$, add C to $\mathsf{XReq}_{\mathsf{PO}}(x)$.

4.5 Downward eventualities

Let $C \in \mathsf{Req}^{\downarrow}(x)$. The request is *fulfilled* if some strict descendant of x already has C in its label. If not, one of the following rules may be applied.

- ($\exists$PPI-realize) Choose a child y of x (existing or fresh) and add C to $L(y)$.
- ($\exists$PPI-defer) Choose a child y of x (existing or fresh) and add C to $\mathsf{Req}^{\downarrow}(y)$.

A fresh child created by either rule is an immediate PPI-successor of x in the cover tree. All current formulas in $\mathsf{All}^{\downarrow}(x)$ are copied to its label and to its own $\mathsf{All}^{\downarrow}$ set.

4.6 Upward eventualities

Let $C \in \mathsf{Req}^{\uparrow}(x)$. The request is *fulfilled* if some strict ancestor of x already has C in its label. If not, one of the following rules may be applied.

- ($\exists$PP-realize) If x has parent p, add C to $L(p)$.
- ($\exists$PP-defer) If x has parent p, add C to $\mathsf{Req}^{\uparrow}(p)$.
- ($\exists$PP-top) If x has no parent, create a fresh parent p above the current ambient root and make p the new ambient root; then either realize C at p or defer it upward at p.

Whenever a fresh parent is created, all formulas in $\mathsf{All}^{\uparrow}(x)$ are inserted into $L(p)$ and into $\mathsf{All}^{\uparrow}(p)$.

Remark 4.2 (Why top extension is necessary). The formula $\exists\mathsf{PP}.C$ is an *upward eventuality*; it does not say that the immediate parent must satisfy C. For example,

$$\exists\mathsf{PP}.C \sqcap \exists\mathsf{PP}.\forall\mathsf{PPI}.\neg C$$

is satisfiable when the distinguished evaluation node lies strictly below an intermediate ancestor satisfying $\forall\mathsf{PPI}.\neg C$ and strictly below a still higher ancestor satisfying C. This is why the calculus allows upward deferment and top extension instead of greedily forcing the current parent to realize C.

4.7 Cross witness obligations

No local rule creates a direct DR- or PO-witness as a tree edge. Instead, the sets $\mathsf{XReq}_{\mathsf{DR}}(x)$ and $\mathsf{XReq}_{\mathsf{PO}}(x)$ are discharged by the finite global side-condition checker of Section 6. The slots $\mathsf{Need}_{\mathsf{DR}}(x)$ and $\mathsf{Need}_{\mathsf{PO}}(x)$ are the dual universal side: they record what every future DR- respectively PO-witness of x must satisfy. Intuitively, $\mathsf{XReq}_{\mathsf{DR}}/\mathsf{XReq}_{\mathsf{PO}}$ become existential witness-menu constraints on the finite quotient, while $\mathsf{Need}_{\mathsf{DR}}/\mathsf{Need}_{\mathsf{PO}}$ become the local type-safety side conditions that those menu entries must meet.

5 Rank-k signatures and blocking

5.1 Local packages

Definition 5.1 (Local package). *For a tableau node x, define*

$$\mathrm{Loc}(x) := (L(x), \mathsf{All}^{\uparrow}(x), \mathsf{All}^{\downarrow}(x), \mathsf{Req}^{\uparrow}(x), \mathsf{Req}^{\downarrow}(x), \mathsf{Need}_{\mathsf{DR}}(x), \mathsf{Need}_{\mathsf{PO}}(x), \mathsf{XReq}_{\mathsf{DR}}(x), \mathsf{XReq}_{\mathsf{PO}}(x), \mathsf{Top}(x), \mathsf{Eval}($$

where $\mathsf{Top}(x)$ *records whether x is the current ambient root and* $\mathsf{Eval}(x)$ *records whether $x = r_*$.*

The set of possible local packages is finite because it is built from subsets of the finite closure $\mathrm{cl}(C_0)$ and two booleans.

Remark 5.2 (Universal side slots versus existential side slots). The components $\mathsf{Need}_{\mathsf{DR}}, \mathsf{Need}_{\mathsf{PO}}$ are universal *expectation* slots: they summarize what every future DR- or PO-related witness must satisfy because of formulas already present in $L(x)$. By contrast, $\mathsf{XReq}_{\mathsf{DR}}, \mathsf{XReq}_{\mathsf{PO}}$ are

existential demand slots: they say that some such witness still has to be found. The calculus keeps both kinds of information explicitly because both become part of the blocking signature.

5.2 Finite signatures

The signature language is inherited from the split-forest note, but now based on local packages rather than plain local types.

Definition 5.3 (Rank-0 signatures)**.** *Set*

$$\Sigma_0 := \{\mathrm{Loc}(x) : x \textit{ may occur in a saturated local package over } C_0\}.$$

Write $\mathrm{State}_0(x) := \mathrm{Loc}(x)$.

Definition 5.4 (Rank-$(k+1)$ signatures)**.** *Assume Σ_k is defined. Then a rank-$(k+1)$ signature of a node x is*

$$\mathrm{State}_{k+1}(x) := (\mathrm{Loc}(x), \mathrm{Par}_k(x), \mathrm{Child}_k(x), \mathrm{Iface}_k(x), \mathrm{Wit}_k(x)),$$

where:

- $\mathrm{Par}_k(x) \in \Sigma_k \cup \{\bot\}$ *is the rank-k signature of the parent of x, or $\bot$ if x is the ambient root;*
- $\mathrm{Child}_k(x) : \Sigma_k \to \{0, 1, 2+\}$ *records whether x has no child, exactly one child, or at least two children of each rank-k signature;*
- $\mathrm{Iface}_k(x)$ *is the finite family of ordered child-pair interface descriptors from the split-forest note, now read over child colors in Σ_k;*
- $\mathrm{Wit}_k(x)$ *is a finite support-closed witness menu indicating how the obligations in $\mathsf{XReq}_{\mathsf{DR}}(x)$, $\mathsf{XReq}_{\mathsf{PO}}(x)$, $\mathsf{Req}^{\uparrow}(x)$, and $\mathsf{Req}^{\downarrow}(x)$ are intended to be discharged at signature level, subject to the type-safety side conditions encoded by $\mathsf{Need}_{\mathsf{DR}}(x)$ and $\mathsf{Need}_{\mathsf{PO}}(x)$.*

Let Σ_{k+1} be the finite set of such signatures. Finally set Σ_d with $d = \mathrm{md}(C_0)$.

Remark 5.5. The value $2+$ means "at least two"; it does *not* mean infinite branching. The infinite part of the eventual model comes from depth in the unfolding, not from infinite multiplicity at one node.

Definition 5.6 (Blocking condition)**.** *Fix any total creation order $\prec$ on tableau nodes. A node x is* blocked *by an earlier node $y \prec x$ if*

$$\mathrm{State}_d(x) = \mathrm{State}_d(y)$$

and y is not blocked. A node is active *if it is not blocked.*

Remark 5.7. Because the rank-d signature already contains the parent summary, eventuality sets, interface summaries, and witness menus, this is not mere type blocking. It is an eventuality-aware global caching condition.

6 Global side-checker

Given a finite saturated blocked tableau $\mathcal{T}$, let $Q(\mathcal{T})$ be the finite quotient whose nodes are the active rank-d signatures. Each quotient node carries:

- its local package,
- its child multiplicity map,
- its ordered sibling-interface descriptors,
- its witness menu.

For local packages $\mathrm{Loc}(x), \mathrm{Loc}(y)$, define the explicit cross-witness type-safety predicates

$$\mathrm{Safe}_{\mathsf{DR}}(x, y) \iff \mathsf{Need}_{\mathsf{DR}}(x) \subseteq L(y) \text{ and } \mathsf{Need}_{\mathsf{DR}}(y) \subseteq L(x),$$

$$\mathrm{Safe}_{\mathsf{PO}}(x, y) \iff \mathsf{Need}_{\mathsf{PO}}(x) \subseteq L(y) \text{ and } \mathsf{Need}_{\mathsf{PO}}(y) \subseteq L(x).$$

The conjunctions are symmetric because both DR and PO are symmetric RCC5 relations. Thus a menu entry proposing a DR-witness between endpoint colors p, q is accepted only if the represented local packages satisfy $\mathrm{Safe}_{\mathsf{DR}}$; similarly for PO.

The global side-checker $\mathsf{Patch}(\mathcal{T})$ accepts if all of the following hold.

(i) Every $\mathsf{XReq}_{\mathsf{DR}}$ and $\mathsf{XReq}_{\mathsf{PO}}$ obligation is supported by the quotient node's witness menu.

(ii) Every proposed DR-menu edge between quotient nodes q, q' satisfies

$$\mathsf{Need}_{\mathsf{DR}}(q) \subseteq L(q') \qquad \text{and} \qquad \mathsf{Need}_{\mathsf{DR}}(q') \subseteq L(q).$$

(iii) Every proposed PO-menu edge between quotient nodes q, q' satisfies

$$\mathsf{Need}_{\mathsf{PO}}(q) \subseteq L(q') \qquad \text{and} \qquad \mathsf{Need}_{\mathsf{PO}}(q') \subseteq L(q).$$

(iv) All ordered child-pair interface descriptors are locally coherent and type-safe.

(v) The quotient satisfies the extendibility conditions needed by the canonical refinement lemmas of the split-forest note, i.e. every finite prefix network built from the quotient has a canonical extendible DR/PO refinement.

A saturated blocked tableau is *open* if it is locally clash-free and $\mathsf{Patch}(\mathcal{T})$ accepts.

Remark 6.1. The tableau is therefore hybrid in the same sense that many global-caching calculi are hybrid: local tree expansion plus a finite global consistency check on the quotient. The side-checker is not extra semantic baggage; it is precisely the finite object required by the split-forest soundness proof.

7 Termination

Proposition 7.1 (Finiteness of signatures)**.** *For fixed C_0, the set Σ_d of rank-d signatures is finite.*

Proof. Σ_0 is finite because each component of a local package, including the explicit $\mathsf{Need_{DR}}/\mathsf{Need_{PO}}$ slots, ranges over subsets of the finite closure $\mathrm{cl}(C_0)$ and two booleans. Inductively, if Σ_k is finite, then Par_k ranges over a finite set, Child_k is a function into $\{0, 1, 2+\}$ over a finite domain, Iface_k ranges over the finite descriptor alphabet from the split-forest note, and Wit_k ranges over finite support-closed menus over finite alphabets. Hence Σ_{k+1} is finite. □

Theorem 7.2 (Termination argument)**.** *Every fair branch construction for the proposed calculus terminates in a finite saturated blocked tableau.*

Argument. Boolean and initialization rules only add information from the finite closure and therefore can fire only finitely often per node. The propagation rules also add only formulas from $\mathrm{cl}(C_0)$ to finite local package components. Fresh-node creation occurs only through $\exists\mathsf{PPI}$- and $\exists\mathsf{PP}$-eventuality steps. By Definition 5.6, every newly created active node must realize a new rank-d signature. By Proposition 7.1, only finitely many active signatures exist. Once a signature repeats, the new node is blocked and no further subtree below it needs to be expanded. Thus only finitely many active nodes can ever be created.

Because requests and obligations are finite subsets of $\mathrm{cl}(C_0)$ and are recorded in the signature, no eventuality can generate infinitely many unblocked fresh nodes of pairwise distinct request profiles. Therefore the fair branch construction must reach a point where all local rules are exhausted and every further fresh node would be immediately blocked. At that point the tableau is saturated and blocked. □

8 Soundness

Theorem 8.1 (Soundness argument)**.** *If the proposed calculus has an open saturated blocked tableau for C_0, then C_0 is satisfiable in a strong-EQ model of $\mathcal{ALCI}_{RCC5}$.*

Argument. Let $\mathcal{T}$ be an open saturated blocked tableau. Form the finite quotient $Q(\mathcal{T})$ of active rank-d signatures. By openness, the local tableau is clash-free and the side-checker $\mathsf{Patch}(\mathcal{T})$ accepts. Thus the quotient carries coherent ordered child-pair descriptors and support-closed witness menus, and every finite prefix network of its split-forest unfolding admits a canonical extendible DR/PO refinement.

Now apply the companion split-forest theorem (the completed sibling-interface note): from such a finite quotient one obtains a canonical weak-EQ split-forest realization, then a strong-EQ model by quotienting the typed EQ-congruence classes [3]. The only thing to check is that the tableau obligations are interpreted correctly in that realization.

- If $C \in \mathsf{All}^{\downarrow}(x)$, then every strict descendant of the corresponding split-forest node satisfies C because the local propagation rules copied C to every child and recursively to every lower request-state.
- If $C \in \mathsf{All}^{\uparrow}(x)$, the same holds for strict ancestors by upward propagation and top extension.
- If $C \in \mathsf{Req}^{\downarrow}(x)$, then either the local tableau already realized C on some chosen child branch, or the witness menu stored in $\mathrm{Wit}_d(x)$ witnesses an infinite descendant branch in the quotient unfolding. Hence some strict descendant satisfies C.

- If $C \in \mathsf{Req}^{\uparrow}(x)$, the symmetric argument uses the ambient-top extension chain above the evaluation node.
- If $C \in \mathsf{XReq}_{\mathsf{DR}}(x)$ or $C \in \mathsf{XReq}_{\mathsf{PO}}(x)$, acceptance of $\mathsf{Patch}(\mathcal{T})$ guarantees a designated witness menu entry compatible both with the sibling-interface descriptors and with the universal side conditions in $\mathsf{Need}_{\mathsf{DR}}(x)$ or $\mathsf{Need}_{\mathsf{PO}}(x)$; the split-forest theorem realizes it by the canonical DR/PO refinement.

Therefore every labelled formula at the evaluation node r_* is satisfied in the resulting strong-EQ model. In particular $C_0 \in L(r_*)$, so the model satisfies C_0. □

9 Completeness

Theorem 9.1 (Completeness argument)**.** *If C_0 is satisfiable in a strong-EQ model of $\mathcal{ALCI}_{RCC5}$, then the proposed calculus has an open branch for C_0.*

Argument. Assume $M, r \models C_0$. By the companion split-forest theorem, choose a weak-EQ split-tree presentation of M with distinguished evaluation node corresponding to r and with finite rank-d quotient data [3]. We guide the tableau nondeterministically by that presentation.

Start with one tableau node r_* labelled by C_0. Whenever a disjunction $D_1 \sqcup D_2$ is encountered, choose a disjunct true at the corresponding point of the presentation. Whenever $\exists\mathsf{PPI}.C$ is initialized, follow the chosen descendant witness path from the presentation: either realize C at the next cover child, or defer the request to the child lying on the presentation's witness branch. Similarly, whenever $\exists\mathsf{PP}.C$ is initialized, either realize C at the current parent in the presentation or defer upward; if the present tableau does not yet contain enough ancestors, use top extension to create them. For $\forall\mathsf{PPI}$ and $\forall\mathsf{PP}$, propagation is forced and matches the cover-tree semantics of the presentation.

For $\exists\mathsf{DR}.C$ and $\exists\mathsf{PO}.C$, do not try to materialize a local tree edge. Instead, record the obligation in $\mathsf{XReq}_{\mathsf{DR}}$ or $\mathsf{XReq}_{\mathsf{PO}}$ and extract the appropriate support-closed witness-menu entry from the finite quotient of the presentation. Because the presentation is a genuine model, every extracted witness endpoint also satisfies the dual universal side conditions encoded by $\mathsf{Need}_{\mathsf{DR}}$ and $\mathsf{Need}_{\mathsf{PO}}$, and because the split-forest theorem provides canonical extendible refinements on all finite prefixes, the global side-checker will succeed on those extracted menus.

Finally, by the finite-index lemma for rank-d signatures, only finitely many active signatures can appear along the guided cover tree. Therefore eventually every newly generated node repeats an earlier rank-d signature and is blocked. The resulting branch is finite, saturated, locally clash-free, and accepted by the side-checker; hence it is open. □

10 The resulting decision procedure

Corollary 10.1 (Decision procedure)**.** *The proposed calculus yields a finite search procedure for concept satisfiability in $\mathcal{ALCI}_{RCC5}$: explore locally saturated blocked cover-tree tableaux and accept iff some branch is open.*

Argument. Termination is given by Theorem 7.2; soundness and completeness are given by Theorems 8.1 and 9.1. Therefore the calculus decides satisfiability. □

11 Discussion

Two features of the proposal are worth emphasizing.

First, the cover tree is not a semantic tree model. It is an operational abstraction of the proper-part order. The actual model remains a complete RCC5-labelled graph, reconstructed from the finite quotient and its sibling-interface descriptors.

Second, the calculus is intentionally asymmetric with respect to PP and PPI because a cover tree chooses one orientation. With downward PPI cover edges, $\exists\mathsf{PPI}.C$ is a descendant eventuality and $\exists\mathsf{PP}.C$ is an ancestor eventuality. Reversing the orientation would reverse those roles. The use of a distinguished evaluation node and top extension is therefore not optional; it is forced by the semantics of transitive PP/PPI over a cover tree.

The chief advantage of the tree abstraction is that it makes the global cross-edge problem finite. After join splitting, PP is no longer an open sibling-interface value; EQ is handled by split mates, DR is rigid on Out zones, and only DR/PO remain open on Front–Front pairs. That is exactly the configuration captured by the side-checker.

12 Conclusion

This note proposes a concrete tableau operationalization of the split-forest proof strategy for $\mathcal{ALCI}_{RCC5}$. The local rules keep only a cover tree and treat PP/PPI as ancestor/descendant eventualities. The non-tree modalities DR and PO are discharged globally by the finite sibling-interface checker. Blocking is equality of eventuality-aware rank-d signatures.

The resulting calculus is best viewed as a *hybrid tableau with global caching*: its local part is tableau-like, its blocking condition is finite-signature based, and its final acceptance condition is exactly the patchwork side theorem needed by the companion split-forest semantics. In that sense it is a natural candidate operational calculus for the finite-quotient decidability proof.

References

[1] Michael Wessel. *Qualitative Spatial Reasoning with the ALCIRCC Family – First Results and Unanswered Questions.* University of Hamburg, Memo No. 324/03, 2003.

[2] OpenAI GPT-5.4 Pro, prompted by Michael Wessel. *Finite Sibling-Interface Descriptors and a Proposed Completion for $\mathcal{ALCI}_{RCC5}$.* Manuscript, April 2026.

[3] OpenAI GPT-5.4 Pro, prompted by Michael Wessel. *Finite Sibling-Interface Descriptors and a Proposed Completion for $\mathcal{ALCI}_{RCC5}$: completed split-forest version with canonical refinement and typed-*EQ *quotienting.* Manuscript, April 2026.